HTTP 완벽 가이드 · Week 5

엔터티와 인코딩

본문은 어떻게 실려오는가 · 15장 · p.393 ~ 425

정우인

메시지는 컨테이너, 엔터티는 화물

HTTP 메시지 = 컨테이너

```
HTTP/1.0 200 OK
```
시작줄

엔터티 = 화물

```
Content-Type: text/plain
Content-Length: 18
```
엔터티 헤더

```
\r\n
```

```
Hi! I'm a message!
```
엔터티 본문

엔터티 = 헤더 + 본문

1 엔터티 헤더

본문을 설명하는 정보
타입 · 길이 · 인코딩 …

2 엔터티 본문

실제로 실어 나르는 데이터
텍스트 · 이미지 · JSON …

→ 메시지는 운반 수단, 실제로 전달하려는 건 엔터티

본문은 날 데이터 — 의미는 헤더가 알려준다

```
HTTP/1.0 200 OK\r\n                    시작줄

Content-Type: text/plain\r\n           헤더
Content-Length: 18\r\n

\r\n                                   빈 줄 = 헤더 끝

Hi! I'm a message!                     여기부터 본문
```

1 가공되지 않은 데이터

본문 자체는 그냥 바이트 덩어리
스스로 무엇인지 말하지 않음

2 의미는 헤더가 설명

`text/plain` → 텍스트로 읽어라
18 → 18바이트만큼 읽어라

3 시작 위치

헤더 끝을 알리는 **빈 CRLF 줄**
바로 다음부터 본문

→ 본문은 그냥 바이트, 어떻게 읽을지는 헤더가 알려준다

Content-Length — 본문이 몇 바이트인지

필수 본문이 있는 메시지라면 반드시 포함 (청크 인코딩으로 보낼 때만 예외)

1 잘림 검출

`Content-Length: 10` 이라고 예고

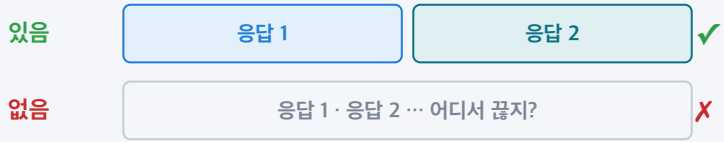


받은 6 < 예고 10 → **잘렸다!**

- 옛날엔 연결 닫힘(FIN) = 응답 끝 → 서버가 죽어도 FIN
- TCP는 받은 건 정확 · 다 받았는지는 모름
- 캐싱 프록시는 **Content-Length 없으면 보통 캐시 X**

2 메시지 경계 구분

하나의 지속 커넥션에 응답이 줄줄이



- 연결이 안 닫히니 FIN으로 끝을 알 수 없음
- 길이를 알아야 다음 메시지 시작을 찾는다
- **청크 인코딩**이면 Content-Length 없이도 가능

→ 본문 길이 하나로 **잘림 검출** + **메시지 경계 구분**

미디어 타입 — 본문이 무엇인지 알려준다

`Content-Type:` **text** / **html**

text — 주 미디어 타입
큰 분류

html — 부 타입
그 안의 구체적인 형식

= HTML 문서
텍스트 중에서도
HTML 형식

text
텍스트

`text/html`
`text/plain`
`text/css`

image
이미지

`image/png`
`image/jpeg`
`image/gif`

audio
오디오

`audio/mpeg`
`audio/ogg`

video
비디오

`video/mp4`
`video/webm`

application
응용 데이터

`application/json`
`application/pdf`

→ **주 타입 / 부 타입** 두 단계로 본문 종류를 알려주고, 받는 쪽은 그걸 보고 처리 방법을 정한다

multipart/form-data — 본문 하나에 여러 조각

```
Content-Type: multipart/form-data; boundary=abc123
```

```
--abc123                                          boundary — 파트 시작
Content-Disposition: form-data; name="title"

내 프로필 사진                                    파트 1 · 텍스트 필드
--abc123                                          boundary — 파트 시작
Content-Disposition: form-data; name="file"; filename="me.png"
Content-Type: image/png

(이미지 바이너리...)                              파트 2 · 파일
--abc123--                                        끝 boundary — 마지막 파트 끝
```

멀티파트란?

- 본문 하나에 **여러 조각(파트)**을 담는 방식
- 각 파트는 **자기 헤더**를 따로 가짐
- 파트 사이는 boundary 문자열로 구분

multipart/form-data

- 폼 제출, 특히 **파일 업로드**에 씀
- **텍스트 필드**와 **파일**을 한 요청에 같이
  예) 제목 + 프로필 사진

→ boundary로 자르고, 파트마다 자기 Content-Type → 성격이 다른 데이터를 한 번에

텍스트 + 파일 하나 받기

FileController.java

```java
@PostMapping(consumes = MULTIPART_FORM_DATA_VALUE)
public String upload(
    @RequestParam("title") String title,
    @RequestParam("file") MultipartFile file) {
  return title + " / "
       + file.getOriginalFilename()
       + " / " + file.getSize() + " bytes";
}
```

실제로 들어오는 요청

```
POST /api/files
Content-Type: multipart/form-data; boundary=XyZ

--XyZ
Content-Disposition: form-data; name="title"

프로필 사진
--XyZ
Content-Disposition: form-data; name="file"; filename="me.png"
Content-Type: image/png

(이미지 바이너리)
--XyZ--
```

MultipartFile 자주 쓰는 메서드

getOriginalFilename()	filename → me.png
getContentType()	파트의 Content-Type → image/png
getSize()	파일 크기
getInputStream() · transferTo()	내용 읽기 · 저장

→ 파트의 name ↔ @RequestParam 이름이 연결 — Spring이 boundary로 나눠서 이름대로 넣어준다

파일 여러 개 받기 — 같은 name은 리스트로

FileController.java

```java
@PostMapping("/multi")
public int uploadMany(
    @RequestParam("files") List<MultipartFile> files) {
  return files.size();  }
```

같은 name="files" 파트를 모아서 주입

List<MultipartFile> files

a.png	b.png	c.pdf
image/png	image/png	application/pdf

실제로 들어오는 요청

```
POST /api/files/multi
Content-Type: multipart/form-data; boundary=XyZ

--XyZ
Content-Disposition: form-data; name="files"; filename="a.png"
Content-Type: image/png

(a.png 바이너리)
--XyZ
Content-Disposition: form-data; name="files"; filename="b.png"
Content-Type: image/png

(b.png 바이너리)
--XyZ
Content-Disposition: form-data; name="files"; filename="c.pdf"
Content-Type: application/pdf

(c.pdf 바이너리)
--XyZ--
```

→ 같은 name="files" 파트가 여러 개면 List<MultipartFile> 로 받는다

콘텐츠 인코딩 — HTTP에선 거의 압축

인코딩 = 데이터를 정해진 규칙에 따라 다른 형태로 바꾸는 것

HTTP 콘텐츠 인코딩은 대부분 **압축** (gzip · br ···)

```
Content-Encoding: gzip
Content-Length: 300
```



1 Content-Length는 압축된 크기

실제로 전송되는 바이트 수 기준
→ 300 = **압축 후** 본문 크기

2 원본 크기를 알려줄 헤더는 없다

압축을 풀기 전엔 원래 크기를 모름
→ 길이 헤더는 전송 기준으로만 존재

→ Content-Encoding = 본문을 어떻게 바꿨나, Content-Length = 바꾼 뒤 크기

압축해서 보내고, 받는 쪽이 푼다



→ Content-Type 은 그대로, Content-Length 는 바뀌고, Content-Encoding 이 붙는다

지금 주로 쓰는 인코딩과 실무 팁

gzip
가장 기본

거의 모든 브라우저 ·
서버가 지원

br
Brotli · 구글

gzip보다 더 작게 압축
브라우저는 HTTPS일 때만 요청

zstd
최근 등장

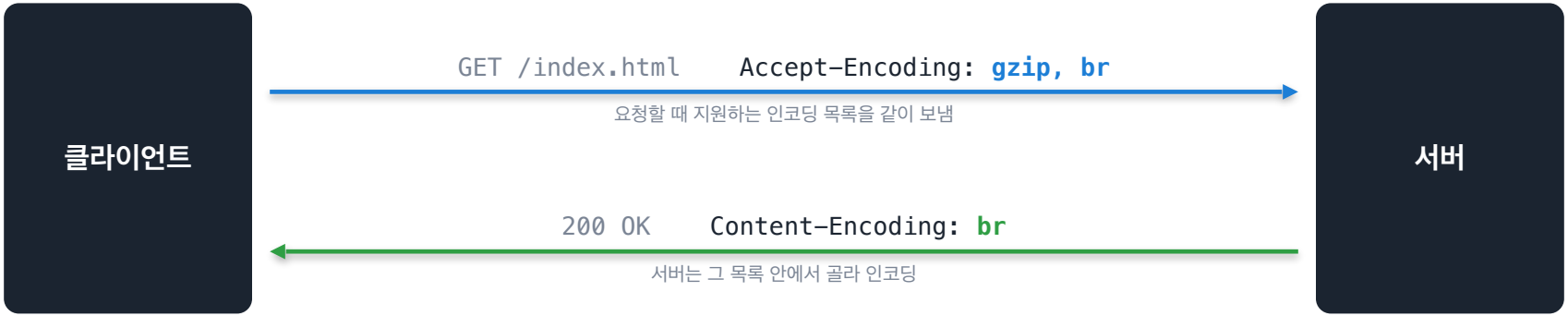
크롬 · 파이어폭스 지원 시작
"이런 것도 있다" 정도

실무에서 알아두면 좋은 것 · 부록

1. **이미 압축된 파일은 또 압축하지 않는다** — png · jpg · mp4 · zip 은 거의 안 줄고 CPU만 씀 → **텍스트(HTML · CSS · JS · JSON)만** 압축
2. **아주 작은 응답도 압축하지 않는다** — 몇백 바이트짜리는 압축해도 이득이 거의 없음
3. **압축은 보통 NGINX에서** — 앞단에서 처리하면 Spring은 신경 쓰지 않아도 됨

→ 기본은 gzip, 요즘 웹은 br · 압축은 텍스트에만

Accept-Encoding — 내가 풀 수 있는 인코딩은 이것



1 왜 필요한가

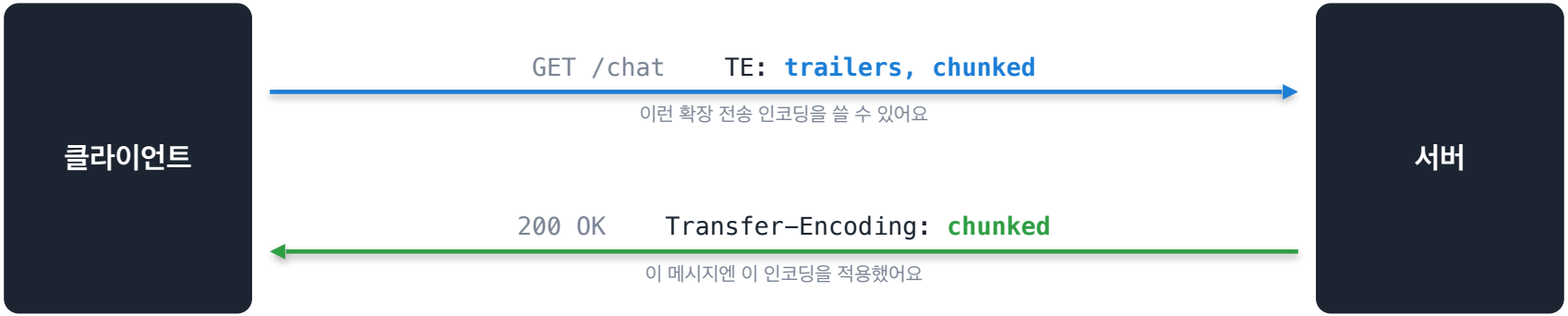
클라이언트가 지원하지 않는 인코딩을
서버가 쓰는 걸 막기 위해

2 헤더가 없으면?

어떤 인코딩이든
다 받는다고 간주

→ 클라이언트는 Accept-Encoding 으로 묻고, 서버는 Content-Encoding 으로 답한다

Transfer-Encoding 과 TE



Transfer-Encoding 응답

메시지에 어떤 전송 인코딩이
적용되었는지 클라이언트에게 알려줌

TE 요청

어떤 확장 전송 인코딩을 쓸 수 있는지
서버에게 알려줌

→ 클라이언트는 TE 로 알리고, 서버는 Transfer-Encoding 으로 답한다

청크 인코딩 — 크기를 모를 때 조각내서 보낸다

1 지속 커넥션에선 길이가 필요

다음 응답과 구분하려면 **Content-Length** 필수

2 그런데 전체 크기를 모른다면?

만들어지는 대로 바로 보내야 할 때
예) AI 채팅 — 답변이 생성되는 중

3 → 청크 인코딩

Transfer-Encoding: chunked
조각마다 크기를 붙여서 보낸다

AI 채팅 응답 — 생성되는 대로 전송



크기 (16진수 바이트 수) + 데이터 · **0** 이면 끝

HTTP 버전별

HTTP/1.1 많이 쓴다
HTTP/2 이상 어차피 프레임 단위로 쪼개 보내니 필요 없음

→ 크기를 몰라도 된다 — 조각마다 크기를 붙이고, 0 으로 끝을 알린다

청크 인코딩된 메시지의 구조

HTTP 응답

```
HTTP/1.1 200 OK<CR><LF>
Content-type: text/plain<CR><LF>
Transfer-encoding: chunked<CR><LF>
Trailer: Content-MD5<CR><LF>
<CR><LF>
```

`chunked` → 조각내서 보낼게 · Content-Length 없음
`Trailer` → 끝에 Content-MD5 붙일게 (예고)
빈 줄 다음부터 본문

청크 #1

```
27<CR><LF>
We hold these truths to be self-evident<CR><LF>
```

크기 한 줄 + 데이터 한 줄
크기는 16진수 · `0x27 = 39` 글자

청크 #2

```
26<CR><LF>
, that all men are created equal, that<CR><LF>
```

`0x26 = 38` 글자

청크 #3

```
84<CR><LF>
 they are endowed by their Creator … Happiness.<CR><LF>
```

`0x84 = 132` 글자

마지막 청크

```
0<CR><LF>
```

`0` = 본문 끝

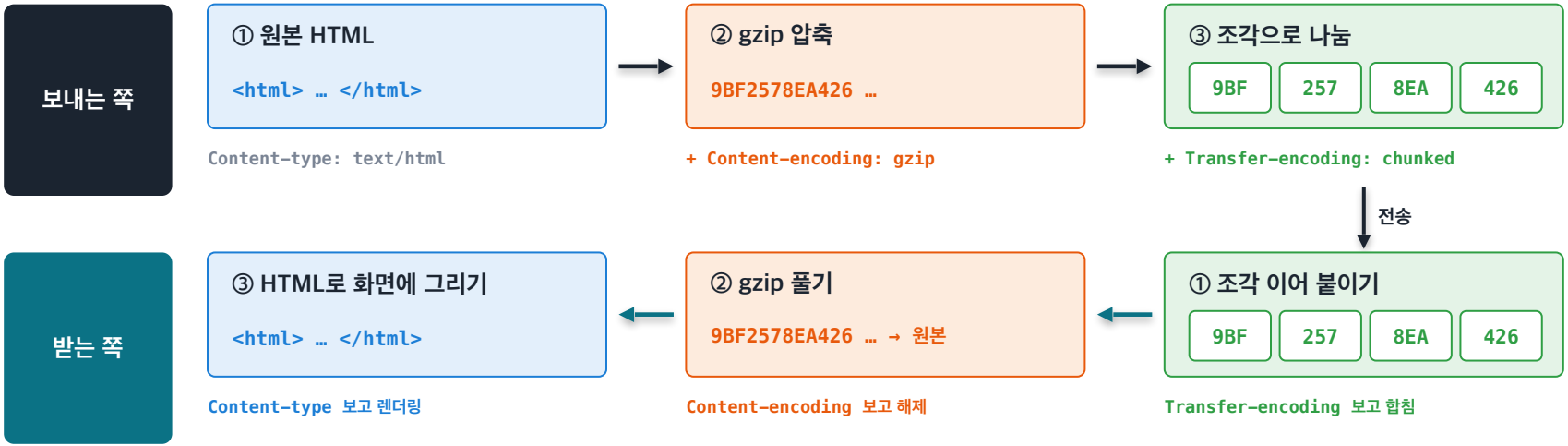
트레일러*

```
Content-MD5: gjqei54p26tjisgj3p4utjgrj53<CR><LF>
```

본문 뒤 추가 헤더 — MD5는 본문 전체를 다 봐야 계산할 수 있어서 뒤에 붙인다

* 메시지 헤더에 Trailer 헤더가 있을 때만 존재한다

gzip으로 압축하고, chunked로 나눠 보낸다



보낼 때 **압축 → 조각** 받을 때 **조각 합치기 → 압축 해제** (순서가 반대)

→ **콘텐츠 인코딩**은 본문을 바꾸고, **전송 인코딩**은 그걸 나눠 실어 나른다

택배로 보면 — 압축팩과 여러 상자

1 콘텐츠 인코딩

물건을 압축팩에 넣어 부피를 줄이는 것

2 전송 인코딩

그 압축팩을 여러 상자에 나눠 배송하는 것

3 받는 사람

상자를 모아 압축팩 복원 → 압축팩을 열어 물건을 꺼냄

왜 이 조합이 흔한가 · 부록

NGINX가 gzip 압축하면서 바로바로 보냄

→ 압축 결과가 몇 바이트일지 **미리 모름**

→ **Content-Length** 못 씀 → **chunked**

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Encoding: gzip
Transfer-Encoding: chunked
```

Content-Length 없음 — gzip 응답에 흔히 보이는 조합

→ 압축팩(gzip)을 여러 상자(chunked)에 — 받을 땐 상자 모으기 → 압축팩 열기